

REDX OpenAPI — Integration Specification & Implementation Brief

Source: REDX Merchant Dashboard → Developer APIs (<https://redx.com.bd/developer-api/>), captured 2026-09-09. Purpose: hand this document to an engineering agent (Claude Code) as the single source of truth for building a **production-grade REDX courier integration** in a Node.js/TypeScript (NestJS) backend. Everything under "Raw API Reference" is transcribed directly from REDX's own docs. Everything under "Engineering Notes" and "Suggested NestJS Implementation Plan" is added analysis/guidance for the implementer — call these out explicitly as *our* recommendations, not REDX's official statements, since REDX's public docs do not cover them.

1. Overview

REDX OpenAPI lets a merchant create and manage parcels, manage pickup stores, look up service areas, calculate delivery charges, and receive parcel-status webhooks. API version in all paths: **v1.0.0-beta**.

Two environments exist. Every environment has its own **base URL** and its own **JWT access token**, generated from the merchant dashboard ("Request token" button under Developer APIs → Configuration). Tokens are merchant-account-specific secrets — REDX's docs do not show real token values, and neither should this document or any code committed to a repo.

Environment	Base URL
Sandbox (testing only, simulated data)	https://sandbox.redx.com.bd/v1.0.0-beta
Production (live data)	https://openapi.redx.com.bd/v1.0.0-beta

REDX explicitly recommends fully testing in Sandbox before switching to Production.

2. Authentication

Every request (except the incoming webhook call from REDX to the merchant) must include:

API-ACCESS-TOKEN: Bearer <jwt_token>

- <jwt_token> is obtained per-environment from the merchant dashboard.
- There is no documented refresh/expiry flow, scopes, or rate-limit header in REDX's public docs — see Engineering Notes §9 for how to handle this defensively.

3. Raw API Reference

All endpoints below are relative to `{{base_url}}` = one of the two base URLs in §1 (already includes `/v1.0.0-beta`, so don't double it).

3.1 Track Parcel

- **Method:** GET
- **Endpoint:** `/parcel/track/{tracking_id}`

```
curl --location '{{base_url}}/parcel/track/{{tracking_id}}' \  
  
--header 'API-ACCESS-TOKEN: Bearer {{jwt_token}}'
```

Path params

Field	Type	Required	Description
tracking_id	string	Yes	Unique identifier assigned to a parcel for tracking.

Sample response

```
{  
  
  "tracking": [  
  
    {  
      "message_en": "Package is created successfully",  
      "message_bn": "পার্সেলটি সফল ভাবে প্লেস করা হয়েছে",  
      "time": "2020-02-04T21:19:41.000Z" },  
  
    {  
      "message_en": "Package is picked up",  
      "message_bn": "পার্সেল পিকাপ সম্পন্ন হয়েছে",  
      "time": "2020-02-05T11:41:03.000Z" }  
  
  ]  
}
```

Field	Type	Optional	Description
tracking	array	No	List of tracking updates for the parcel.
tracking[].message_en	string	No	Tracking update message in English.
tracking[].message_bn	string	No	Tracking update message in Bengali.
tracking[].time	string (ISO 8601)	No	Timestamp of the tracking update.

3.2 Get Parcel Details

- **Method:** GET
- **Endpoint:** `/parcel/info/{tracking_id}`

```
curl --location '{{base_url}}/parcel/info/{{tracking_id}}' \
--header 'API-ACCESS-TOKEN: Bearer {{jwt_token}}'
```

Path params

Field	Type	Required	Description
tracking_id	string	Yes	Unique identifier assigned to a parcel for tracking.

Sample response

```
{
  "parcel": {
    "tracking_id": "21A427TU4BN3R",
```

```
"customer_address": "Test Address",

"delivery_area": "Mirpur DOHS",

"delivery_area_id": 12,

"charge": 60,

"customer_name": "Test Customer",

"customer_phone": "01987654321",

"cash_collection_amount": 13293,

"parcel_weight": 500,

"merchant_invoice_id": "ACBD1234TEST",

"status": "pickup-pending",

"instruction": "",

"created_at": "2021-04-27T08:29:14.000Z",

"delivery_type": "regular",

"value": "0",

"pickup_location": {

  "id": 1,

  "name": "Malibag",

  "address": "Malibagh",

  "area_name": "Malibag",

  "area_id": 1

}

}
```

}

Field	Type	Optional	Description
parcel	object	No	Parcel details.
parcel.tracking_id	string	No	Unique tracking ID of the parcel.
parcel.customer_address	string	No	Delivery address of the customer.
parcel.delivery_area	string	No	Name of the delivery area.
parcel.delivery_area_id	number	No	Unique ID of the delivery area.
parcel.charge	number	No	Delivery charge for the parcel.
parcel.customer_name	string	No	Name of the customer receiving the parcel.
parcel.customer_phone	string	No	Phone number of the customer.
parcel.cash_collection_amount	number	No	Amount to be collected from the customer.
parcel.parcel_weight	number	No	Weight of the parcel in grams .
parcel.merchant_invoice_id	string	No	Invoice ID provided by the merchant.
parcel.status	string	No	Current status of the parcel (see §3.11 status table).
parcel.instruction	string	Yes	Additional delivery instructions (if any).

Field	Type	Optional	Description
parcel.created_at	string (ISO 8601)	No	Timestamp when the parcel was created.
parcel.delivery_type	string	No	Type of delivery (see §3.12 delivery-type table).
parcel.value	string	No	Declared value of the parcel.
parcel.pickup_location	object	No	Pickup location details.
parcel.pickup_location.id	number	No	Unique ID of the pickup location.
parcel.pickup_location.name	string	No	Name of the pickup location.
parcel.pickup_location.address	string	No	Address of the pickup location.
parcel.pickup_location.area_name	string	No	Area name of the pickup location.
parcel.pickup_location.area_id	number	No	Unique ID of the pickup location's area.

3.3 Create Parcel

- **Method:** POST
- **Endpoint:** /parcel

```
curl --location '{{base_url}}/parcel' \

--header 'API-ACCESS-TOKEN: Bearer {{jwt_token}}' \

--header 'Content-Type: application/json' \

--data '{
```

```
"customer_name": "{{customer_name}}",

"customer_phone": "{{customer_phone}}",

"delivery_area": "{{delivery_area}}",

"delivery_area_id": {{delivery_area_id}},

"customer_address": "{{customer_address}}",

"merchant_invoice_id": "{{merchant_invoice_id}}",

"cash_collection_amount": "{{cash_collection_amount}}",

"parcel_weight": {{parcel_weight}},

"instruction": "{{instruction}}",

"value": {{value}},

"is_closed_box": "{{is_closed_box}}",

"pickup_store_id": {{pickup_store_id}},

"parcel_details_json": [

    { "name": "{{name}}", "category": "{{category}}", "value": {{value}} },

    { "name": "{{name}}", "category": "{{category}}", "value": {{value}} }

]

}'
```

Body params

Field	Type	Required	Description
customer_name	string	Yes	Full name of the customer receiving the parcel.

Field	Type	Required	Description
customer_phone	string	Yes	Contact phone number of the customer.
delivery_area	string	Yes	Name of the delivery area.
delivery_area_id	integer	Yes	Unique identifier of the delivery area (from /areas).
customer_address	string	Yes	Complete delivery address.
cash_collection_amount	string	Yes	Amount to collect from customer on delivery (COD amount; 0 for prepaid).
parcel_weight	string*	Yes	Weight of the parcel. *Docs say "appropriate units (e.g., kg, g)" here but the Get Parcel Details response returns this in grams as a number — see Engineering Note §9.1, confirm unit in Sandbox before trusting either doc text.
merchant_invoice_id	string	No	Merchant's own invoice/order ID for reference/reconciliation.
instruction	string	No	Special delivery instructions.

Field	Type	Required	Description
type	string	No	Parcel type, "mainly used for reverse shipments." No enum of accepted values documented — see Engineering Note §9.2.
value	string	Yes	Declared value of the parcel (for compensation calc).
parcel_details_json	array of objects	No	Line-item breakdown of parcel contents (see below).
pickup_store_id	string	No	ID of the pickup store to collect from (from Create/Get Pickup Store).
is_closed_box	—	Not documented in the params table, but present in REDX's own sample request body. Treat as optional/boolean-ish; verify meaning and accepted values in Sandbox — see Engineering Note §9.3.	

Parcel Details Object (`parcel_details_json[]`)

Field	Type	Required	Description
name	string	Yes	Name of the item inside the parcel.

Field	Type	Required	Description
category	string	Yes	Category of the item (e.g., electronics, clothing).
value	integer	Yes	Declared value of the individual item.

Sample response

```
{ "tracking_id": "20A312THJDJ8" }
```

Field	Type	Optional	Description
tracking_id	string	No	Unique tracking ID of the newly created parcel.

3.4 Update Parcel

- **Method:** PATCH
- **Endpoint:** `/parcels` —  note the plural, unlike every other parcel endpoint which uses singular `/parcel`. This is exactly as published by REDX; do not "fix" it to singular without testing in Sandbox first (see Engineering Note §9.4).

```
curl --location --request PATCH '{{base_url}}/parcels' \
  --header 'API-ACCESS-TOKEN: Bearer {{jwt_token}}' \
  --header 'Content-Type: application/json' \
  --data '{
    "entity_type": "parcel-tracking-id",
    "entity_id": "{{tracking_id}}",
    "update_details": {
```

```

    "property_name": "status",

    "new_value": "cancelled",

    "reason": "Your Cancellation Reason"

  }

}'

```

Body params

Field	Type	Required	Description
entity_type	string	Yes	Type of entity being updated, e.g. <code>"parcel-tracking-id"</code> .
entity_id	string	Yes	Unique identifier of the parcel to update (the tracking ID).
update_details	object	Yes	Object describing the update.
update_details.property_name	string	Yes	Name of the parcel property to update (e.g. <code>status</code> , <code>delivery_address</code>). No exhaustive list of updatable properties or allowed <code>new_values</code> is published — see Engineering Note §9.5.
update_details.new_value	string	Yes	New value for the property.

Field	Type	Required	Description
update_details.reason	string	No	Optional reason, useful for audit logging.

Sample response

```
{ "success": true, "message": "Request Accepted" }
```

Field	Type	Optional	Description
success	boolean	No	Whether the request was accepted.
message	string	No	Human-readable status message.

Note: **"Request Accepted"** suggests this may be processed asynchronously rather than applied synchronously — treat the update as "accepted for processing," and re-fetch parcel details or wait for a webhook to confirm the actual state change.

3.5 Get Areas

- **Method:** **GET**
- **Endpoint:** **/areas**

```
curl --location '{{base_url}}/areas' \
```

```
--header 'API-ACCESS-TOKEN: Bearer {{jwt_token}}'
```

Sample response

```
{
  "areas": [
    { "id": 1, "name": "Mohammadpur(Dhaka)", "post_code": 1207, "division_name": "Dhaka",
      "zone_id": 1 },
```

```
{ "id": 2, "name": "Dhanmondi", "post_code": 1209, "division_name": "Dhaka", "zone_id": 1 },
{ "id": 3, "name": "Gulshan", "post_code": 1212, "division_name": "Dhaka", "zone_id": 1 }
]
}
```

Field	Type	Optional	Description
areas	array	No	List of areas available for delivery.
areas[].id	number	No	Unique identifier of the area — this is what <code>delivery_area_id</code> / <code>pickup_area_id</code> refer to elsewhere.
areas[].name	string	No	Name of the area.
areas[].post_code	number	No	Postal code of the area.
areas[].division_name	string	No	Division the area belongs to.
areas[].zone_id	number	No	Zone identifier for the area.

No pagination parameters are documented for this endpoint, and it returns the full national area list — cache this response (see Engineering Note §9.6).

3.6 Get Areas by Postal Code

- **Method:** `GET`
- **Endpoint:** `/areas?post_code={postal_code}`

```
curl --location '{{base_url}}/areas?post_code={{post_code}}' \
```

```
--header 'API-ACCESS-TOKEN: Bearer {{jwt_token}}'
```

Field	Type	Required	Description
post_code	integer	Yes	Postal code to filter areas by.

Response shape identical to §3.5, filtered to that postal code.

3.7 Get Areas by District Name

- **Method:** GET
- **Endpoint:** /areas?district_name={district_name}

```
curl --location '{{base_url}}/areas?district_name={{district_name}}' \
--header 'API-ACCESS-TOKEN: Bearer {{jwt_token}}'
```

Field	Type	Required	Description
district_name	string	Yes	District name to filter areas by.

Response shape identical to §3.5, filtered to that district. (Note: response field is `division_name`, but the filter query param is `district_name` — REDX appears to use "district" and "division" interchangeably here; confirm accepted values against the `division_name` values returned by §3.5.)

3.8 Create Pickup Store

- **Method:** POST
- **Endpoint:** /pickup/store

```
curl --location '{{base_url}}/pickup/store' \
--header 'API-ACCESS-TOKEN: Bearer {{jwt_token}}' \
--data '{
```

```
"name": "{{name}}",  
  
"phone": "{{phone}}",  
  
"address": "{{address}}",  
  
"area_id": {{area_id}}  
  
}'
```

Field	Type	Required	Description
name	string	Yes	Name of the pickup store.
phone	string	Yes	Contact phone number for the pickup store.
address	string	Yes	Physical address of the pickup store.
area_id	integer	Yes	Area ID (from /areas) where the pickup store is located.

Sample response

```
{  
  
  "id": 1,  
  
  "name": "Test Pickup Store",  
  
  "address": "Test Address",  
  
  "area_name": "Mohammadpur(Dhaka)",  
  
  "area_id": 1,  
  
  "phone": "8801898000999"
```

```
}
```

Note the request's **Content-Type: application/json** header is absent from REDX's own sample curl for this endpoint (present on Create Parcel and Update Parcel, missing here and on Get Pickup Stores/Details). Send it anyway — this is very likely just an omission in REDX's sample, not a real requirement to skip it.

3.9 Get Pickup Stores

- **Method:** GET
- **Endpoint:** /pickup/stores

```
curl --location '{{base_url}}/pickup/stores' \  
  
--header 'API-ACCESS-TOKEN: Bearer {{jwt_token}}'
```

Sample response

```
{  
  
  "pickup_stores": [  
  
    { "id": 1, "name": "...", "address": "...", "area_name": "...", "area_id": 1, "phone": "...",  
      "created_at": "..." }  
  
  ]  
  
}
```

No pagination documented.

3.10 Get Pickup Store Details

- **Method:** GET
- **Endpoint:** /pickup/store/info/{pickup_store_id}

```
curl --location '{{base_url}}/pickup/store/info/{{pickup_store_id}}' \  
  
--header 'API-ACCESS-TOKEN: Bearer {{jwt_token}}'
```

Field	Type	Required	Description
pickup_store_id	integer	Yes	Unique identifier of the pickup store.

Sample response

```
{
  "pickup_store": {
    "id": 1,
    "name": "Test Pickup Store",
    "address": "Test Address",
    "area_name": "Mohammadpur(Dhaka)",
    "area_id": 1,
    "phone": "8801898000999",
    "created_at": "2021-09-13T10:39:15.000Z"
  }
}
```

3.11 Calculate Parcel Charge

- **Method:** GET
- **Endpoint:** /charge/charge_calculator

curl --location

```
'{{base_url}}/charge/charge_calculator?delivery_area_id={{delivery_area_id}}&pickup_area_id={
{pickup_area_id}}&cash_collection_amount={{cash_collection_amount}}&weight={{weight}}' \
```

```
--header 'API-ACCESS-TOKEN: Bearer {{jwt_token}}'
```

Field	Type	Required	Description
delivery_area_id	number	Yes	Area ID where the parcel will be delivered.
pickup_area_id	number	Yes	Area ID where the parcel will be picked up.
cash_collection_amount	number	Yes	Total COD amount to collect on delivery.
weight	number	Yes	Weight of the parcel in grams.

Sample response

```
{ "deliveryCharge": 60, "codCharge": 0 }
```

Field	Type	Optional	Description
deliveryCharge	number	No	Delivery cost.
codCharge	number	No	Cash-on-delivery charge.

Use this before Create Parcel to show the customer/merchant the expected cost, and to sanity-check the COD amount you're about to submit.

4. Webhooks

4.1 Configuration

Merchants register one callback URL from the dashboard (Developer APIs → Webhook → "Enter your callback URL" → Save Link). REDX POSTs parcel status updates to that URL as they happen.

4.2 Callback URL Requirements

- Must be **publicly accessible** (REDX's servers must be able to reach it — no localhost/private IPs in production).
- Must accept **POST** requests.
- **Any auth credential your endpoint needs must be embedded in the URL's query string**, since REDX does not support custom headers or a signing secret for the callback. Example: <https://example.com/callback?token=<token>> → Practical implication: treat that query-string token as a shared secret and validate it on every inbound call; rotate it if it ever leaks (e.g., in logs). See Engineering Note §9.7 — since REDX offers no HMAC/signature scheme, this URL token is your *only* authenticity check, so protect it accordingly (don't log full URLs with the token, use HTTPS only, etc.).

4.3 Webhook Payload

Content-Type: [application/json](#)

```
{  
  
  "tracking_number": "<REDX_TRACKING_ID>",  
  
  "timestamp": "<TIMESTAMP>",  
  
  "status": "<STATUS>",  
  
  "message_en": "<MESSAGE_EN>",  
  
  "message_bn": "<MESSAGE_BN>",  
  
  "invoice_number": "<INVOICE_NUMBER>",  
  
  "delivery_type": "<DELIVERY_TYPE>"  
}
```

Your webhook handler should respond quickly (e.g., [200 OK](#) immediately after persisting the event) and process/dispatch business logic asynchronously — REDX's docs don't specify a retry policy or timeout, so assume one may exist and avoid slow synchronous work in the handler (see Engineering Note §9.8).

4.4 Status Values (`status` field)

Status	Meaning
<code>ready-for-delivery</code>	Parcel received from merchant
<code>delivery-in-progress</code>	Parcel dispatched to rider
<code>delivered</code>	Parcel delivered by rider
<code>agent-hold</code>	Parcel on hold with agent
<code>agent-returning</code>	Parcel return in progress
<code>returned</code>	Parcel returned
<code>agent-area-change</code>	Area change requested & in progress
<code>paid</code>	Parcel amount paid (COD remitted to merchant)

Note: the Get Parcel Details sample (§3.2) shows a parcel with `status: "pickup-pending"`, which is **not** in this webhook status table — meaning the full status vocabulary used on parcel objects is larger than the subset of statuses that trigger webhooks. Don't assume the webhook table above is the complete enum for the `parcel.status` field returned by other endpoints (see Engineering Note §9.9).

4.5 Delivery Types (`delivery_type` field)

Delivery Type	Meaning
<code>regular</code>	Regular forward delivery
<code>reverse</code>	Regular reverse delivery
<code>exchange-delivery</code>	Forward exchange parcel
<code>exchange-return</code>	Reverse exchange parcel
<code>partial-delivery</code>	Partial delivery parcel
<code>partial-return</code>	Partial return parcel

5. Platform Plugins (informational — not needed for a custom backend integration)

REDX also offers a WordPress plugin ("View on WordPress") and an official Shopify app ("View on Shopify"), each gated behind their own dashboard token/enable toggle. These are out of scope for a custom NestJS integration and are mentioned here only for completeness.

6. Things NOT documented by REDX that a production integration must still handle

REDX's public docs do not specify: HTTP status codes / error response schema for failures (invalid token, validation errors, not-found, etc.), rate limits, token expiry/refresh, idempotency behavior on duplicate Create Parcel calls, webhook retry/backoff policy or signature verification, or pagination for `/areas` and `/pickup/stores`. Treat every one of these as unknowns to be **discovered empirically against the Sandbox environment** before writing production code that assumes specific behavior — see the Engineering Notes below for how to handle each defensively.

7. Engineering Notes (gaps & recommendations — ours, not REDX's)

9.1 **parcel_weight unit ambiguity**. Create Parcel's param table says "appropriate units (e.g., kg, g)"; Get Parcel Details returns `parcel_weight: 500` for what is presumably ~500g. Resolve by creating a sandbox parcel with a known weight and reading it back via Get Parcel Details before trusting either doc line. Store weight internally as grams (integer) to avoid ambiguity in your own DB regardless of what the wire format turns out to be.

9.2 **type field on Create Parcel** has no documented enum. Don't hardcode assumed values ("`reverse`", etc.) without confirming against Sandbox or REDX support; make it an optional passthrough string in your DTO rather than a strict enum until confirmed.

9.3 **is_closed_box** appears only in REDX's sample body, not in the params table. Treat as optional/undocumented; confirm accepted values (likely "`1`" / "`0`" or boolean) via Sandbox testing, and don't block parcel creation on it.

9.4 Update Parcel path is plural (/parcels) while all other parcel routes are singular. This is copied verbatim from REDX's own docs — implement exactly as published, and add an integration test that hits Sandbox to confirm the literal path works, rather than "correcting" it to `/parcel`.

9.5 Updatable properties for `update_details.property_name` aren't enumerated (only `status` is shown, with `delivery_address` mentioned as another example). Don't build a closed enum in your DTO; accept any string but log/alert on any value your Sandbox testing hasn't specifically verified to work, so unsupported updates fail loudly rather than silently.

9.6 Cache `/areas`. It looks like a full, mostly-static national area list. Fetch it once at startup / via a scheduled job (e.g., daily), cache it (Redis, since it's already in your stack), and serve area lookups (by postal code, by district) from that cache rather than hitting REDX per user request. Provide a manual "refresh areas" admin action for when REDX adds new service areas.

9.7 Webhook callback URL security. Since the only credential mechanism is a token in the URL query string, generate a long random token per-environment, store it as a secret, mount the webhook route only under HTTPS, verify the token constant-time before processing, and never log the full incoming URL (log a redacted version). Also validate the payload shape defensively — this endpoint is technically a public URL with a bearer-in-query-string, so don't trust `tracking_number` blindly; look it up against parcels you actually created before acting on it.

9.8 Webhook handler must be fast and idempotent. Persist the raw event first (with a unique constraint on something like `(tracking_number, status, timestamp)` to dedupe REDX retries/duplicates), return `200` immediately, then process business-side effects (order status sync, notifications, ledger updates) in a background job (BullMQ, which is already in your stack) rather than inline.

9.9 `status` enum used on parcel objects is larger than the webhook status table. At minimum you've seen `pickup-pending` (from Get Parcel Details) in addition to the 8 webhook statuses. Model your internal `ParcelStatus` as an open string with a curated set of "known" statuses you branch on, plus a fallback/"unknown status, log and alert" path — don't make it a strict TypeScript union that throws on an unrecognized value.

9.10 No documented error schema. Build a thin REDX API client wrapper that normalizes whatever error shape comes back (likely varies by failure type) into your own internal error type, log the raw response body+status on failure for later analysis, and add typed error classes (e.g., `RedxAuthError`, `RedxValidationError`, `RedxNotFoundError`, `RedxUnknownError`) that your services can catch distinctly — inferred from actual Sandbox error responses you capture during implementation, not guessed.

9.11 **No documented rate limits or token refresh.** Implement conservative defaults defensively: a per-second/per-minute client-side rate limiter and exponential-backoff retry with jitter on 429/5xx/network errors (skip retrying on 4xx validation errors), and centralize the token so rotating it later means changing one config value / secret, not a code change.

9.12 **Currency/amounts** (`cash_collection_amount`, `value`, `charge`, `deliveryCharge`, `codCharge`) are implicitly BDT with no explicit currency field — don't build in multi-currency support, but do keep amounts as integers (paisa-free, i.e. whole taka, based on the samples) in your DB to match REDX's apparent convention, and confirm in Sandbox whether decimals are accepted.

8. Suggested NestJS Implementation Plan

Given the existing stack (NestJS, TypeScript, PostgreSQL/Prisma, Redis, BullMQ), a clean module shape:

RedxModule (feature module), containing:

- `redx.config.ts` — typed config (base URL per env via `NODE_ENV`/a dedicated `REDX_ENV` var, token from secret store, webhook token secret). Never commit tokens; load from env vars / secrets manager.
- `redx-http.client.ts` — a thin wrapper (e.g. around `@nestjs/axios` or `fetch`) that: injects the `API-ACCESS-TOKEN` header, applies the retry/backoff + rate-limit policy (Note 9.11), normalizes errors into `RedxApiException` subtypes (Note 9.10), and logs request/response metadata (redacting the token) for observability.
- `redx.service.ts` — one method per endpoint in §3 (`trackParcel`, `getParcelInfo`, `createParcel`, `updateParcel`, `getAreas/getAreasByPostCode/getAreasByDistrict`, `createPickupStore`, `getPickupStores`, `getPickupStoreInfo`, `calculateCharge`), each typed with a request DTO (`class-validator` decorated) and a response interface matching §3's tables.
- `dto/` — `CreateParcelDto`, `UpdateParcelDto`, `ParcelDetailItemDto`, etc., validated with `class-validator/class-transformer`, matching the Required/Optional columns above.
- `areas-cache.service.ts` — loads/caches `/areas` in Redis per Note 9.6, exposes lookups by id/postcode/district, with a scheduled refresh (`@nestjs/schedule`) and a manual admin refresh endpoint.
- `redx-webhook.controller.ts` — `POST /webhooks/redx?token=...` (or your chosen path), validates the query token first (Note 9.7), validates payload shape, persists the raw event with a dedupe constraint (Note 9.8), enqueues a BullMQ job for downstream processing, returns `200` fast.

- `redx-webhook.processor.ts` — BullMQ processor that maps `status/delivery_type` (§4.4/§4.5) to your internal order/shipment states, updates your DB, and triggers any customer-facing notifications.
- Persist your own `Parcel/Shipment` entity keyed by `merchant_invoice_id` (your own order ID) ↔ REDX `tracking_id`, so both directions of lookup are cheap.
- Add integration tests that run against **Sandbox** (never Production) for: create → track → get-info → simulate webhook → update/cancel, gated behind an env flag so they don't run against real REDX in CI without credentials.
- Add a runbook/README section documenting: how to get a Sandbox token, how to switch envs, what the webhook URL should be per environment (e.g. staging vs prod), and the open questions from §7 that were resolved during implementation (record the actual observed behavior once verified, so this doc can be updated with confirmed facts instead of caveats).

9. Suggested Build Order for Claude Code

1. Config + typed HTTP client + error normalization (§8, first two bullets).
2. Areas endpoints + Redis cache (read-only, low risk, good smoke test of auth).
3. Charge calculator (read-only).
4. Pickup store CRUD-ish endpoints (create/list/detail).
5. Create Parcel + Get Parcel Details + Track Parcel (the core flow) — verify the weight-unit and `is_closed_box/type` open questions (§7 9.1–9.3) against Sandbox while building this.
6. Update Parcel — verify the `/parcels` path and which `property_name/new_value` combos actually work against Sandbox (§7 9.4–9.5).
7. Webhook receiver + processor, with the security/idempotency handling from §7 9.7–9.9.
8. Integration tests against Sandbox for the full lifecycle; only then wire in Production credentials behind an explicit env switch.